> Neka je dat pozitivan realan broj x sa konačnim decimalskim zapisom i neka su dati prirodni brojevi n i m, tako da $n<m$. Formirati niz razlomaka p/q takvih da za imenilac q važi $n<=q<=m$ (tj. $q=n,n+1,...,m$) i pri tom imeniocu q pridružujemo brojilac p koji određujemo zaokruživanjem na najbliži priodan broj proizvoda $x \cdot q$. Predstaviti svaki razlomak p/q u obliku verižnog razlomaka. U nizu razlomaka p/q izdvojiti:
>
> - najbolje racionalne aproksimacije I vrste,
> - najbolje racionalne aproksimacije II vrste,
> - sortirati sve razlomke p/q po uslovu minimalnosti apsolutne greske $|x-p/q|$.

Napisani kod ćemo testirati na primjeru konstante $v=log_2(3)-1=0.5849625007211561815...$, a za parametre n i m uzeti brojeve 7 i 53 respektivno.

In [6]:
```
from math import log

v = log(3, 2) - 1
n, m = 7, 53
```

Prvi korak je pisanje pomoćnih funkcija, koje će olakšati generisanje verižnog oblika razlomaka i utvrđivanje "kvaliteta" razlomaka.

Prva funkcija (*generate_continued_fraction*) će generisati verižni oblik razlomka za dato p i q. Naredne dvije funkcije (*is_fst_deg_best* i *is_snd_deg_best*) utvrđuju da li se radi o najboljoj racionalnoj aproksimaciji I i II vrste.

In [7]:
```
def generate_simple_continued_fraction(p: int, q: int) -> list[int]:
        a = [0]
        a[0], p = divmod(p, q)

        while p != 0:
                quotient, remainder = divmod(q, p)
                a.append(quotient)
                p, q = remainder, p

        return a
```

```
def is_fst_deg_best(x: float, p: int, q: int) -> bool:
        for s in range(1, q):
                r = round(s * x)
                if r / s == p / q:
                        continue

                if abs(x - r / s) < abs(x - p / q):
                        return False

        return True

def is_snd_deg_best(x: float, p: int, q: int) -> bool:
        for s in range(1, q):
                r = round(s * x)
                if r / s == p / q:
                        continue

                if abs(s * x - r) < abs(q * x - p):
                        return False

        return True
```

Nakon pisanja pomoćnih funkcija, možemo preći na dio koda koji prolazi kroz svako moguće q i generiše razlomke. A usput, za svaki razlomak, utvrđujemo da li se radi o najboljoj racionalnoj aproksimaciji.

Sve razlomke ćemo čuvati u listi, kako bismo ih mogli kasnije i sortirati i ispisati.

In [8]:
```
from dataclasses import dataclass

@dataclass(order=True)
class RationalApproximation:
        p: int
        q: int
        a: list[int]

        first_degree: bool
        second_degree: bool

        def __float__(self) -> float:
```

```
                return self.p / self.q

fractions: list[RationalApproximation] = []
for q in range(n, m + 1):
        p = round(q * v)

        a = generate_simple_continued_fraction(p, q)
        first_degree = is_fst_deg_best(v, p, q)
        second_degree = is_snd_deg_best(v, p, q) if first_degree else False

        fractions.append(RationalApproximation(
                p, q,
                a, first_degree, second_degree
        ))
```

Potom je potrebno isfiltrirati i sortirati razlomke. Prema tekstu zadatka, sortiranje se vrši po rastućoj vrijednosti apsolutne greške. Dok, kada je riječ o filtriranju, potrebno je ukloniti razlomke koji su u suštini isti, tj. one koji se pojavljuju i u nekom skraćenom obliku (npr. *7/12* i *14/24*).

In [9]:
```
def simplify_fraction(p: int, q: int) -> tuple[int, int]:
    i = 2
    while i < min(p, q) + 1:
        if p % i == 0 and q % i == 0:
            p = p // i
            q = q // i
        else:
            i += 1
    return p, q

unique_fractions: dict[tuple[int, int], list[RationalApproximation]] = {}
for frac in fractions:
    simplified = simplify_fraction(frac.p, frac.q)
    if simplified not in unique_fractions:
        unique_fractions[simplified] = []
    unique_fractions[simplified].append(frac)

fractions = [min(group) for group in unique_fractions.values()]
fractions.sort(key=lambda r: abs(r.p / r.q - v))
```

Ssada imamo sve informacije da ispišemo traženu tabelu. Svaki razlomak je napisan u 3 oblika:

1. kao razlomak
2. kao realan broj
3. kroz verižni zapis

Različite boje određuju "kvalitet" aproksimacije:

- **Crvena** boja - Najbolja racionalna aproksimacija I vrste
- **Plava** boja - Najbolja racionalna aproksimacija II vrste
- **Bijela** boja - Ostali razlomci

In [10]:
```
from termcolor import colored

def format_simple_continued_fraction(a: list[int]) -> str:
        if len(a) == 1:
                return f"[{a[0]}]"

        return f"[{a[0]}; {', '.join(map(str, a[1:]))}]"

for fraction in fractions:
        if fraction.second_degree:
                color = "red"
        elif fraction.first_degree:
                color = "blue"
        else:
                color = "light_grey"

        ratio_format = f"{fraction.p}/{fraction.q}"
        real_format = f"{float(fraction):.9f}"
        difference = f"{v - float(fraction):+.9f}"
        array_format = format_simple_continued_fraction(fraction.a)

        text = f"{ratio_format} = {real_format} ({difference}), {array_format}"

        print(colored(text, color))
```

31/53 = 0.584905660 (+0.000056840), [0; 1, 1, 2, 2, 4]
24/41 = 0.585365854 (-0.000403353), [0; 1, 1, 2, 2, 3]
17/29 = 0.586206897 (-0.001244396), [0; 1, 1, 2, 2, 2]
7/12 = 0.583333333 (+0.001629167), [0; 1, 1, 2, 2]
27/46 = 0.586956522 (-0.001994021), [0; 1, 1, 2, 2, 1, 2]
10/17 = 0.588235294 (-0.003272793), [0; 1, 1, 2, 3]
25/43 = 0.581395349 (+0.003567152), [0; 1, 1, 2, 1, 1, 3]
18/31 = 0.580645161 (+0.004317339), [0; 1, 1, 2, 1, 1, 2]
23/39 = 0.589743590 (-0.004781089), [0; 1, 1, 2, 3, 2]
29/50 = 0.580000000 (+0.004962501), [0; 1, 1, 2, 1, 1, 1, 2]
13/22 = 0.590909091 (-0.005946590), [0; 1, 1, 2, 4]
11/19 = 0.578947368 (+0.006015132), [0; 1, 1, 2, 1, 2]
29/49 = 0.591836735 (-0.006874234), [0; 1, 1, 2, 4, 2]
26/45 = 0.577777778 (+0.007184723), [0; 1, 1, 2, 1, 2, 2]
16/27 = 0.592592593 (-0.007630092), [0; 1, 1, 2, 5]
15/26 = 0.576923077 (+0.008039424), [0; 1, 1, 2, 1, 3]
19/32 = 0.593750000 (-0.008787499), [0; 1, 1, 2, 6]
19/33 = 0.575757576 (+0.009204925), [0; 1, 1, 2, 1, 4]
22/37 = 0.594594595 (-0.009632094), [0; 1, 1, 2, 7]
23/40 = 0.575000000 (+0.009962501), [0; 1, 1, 2, 1, 5]
25/42 = 0.595238095 (-0.010275595), [0; 1, 1, 2, 8]
27/47 = 0.574468085 (+0.010494416), [0; 1, 1, 2, 1, 6]
4/7 = 0.571428571 (+0.013533929), [0; 1, 1, 3]
6/10 = 0.600000000 (-0.015037499), [0; 1, 1, 2]
13/23 = 0.565217391 (+0.019745109), [0; 1, 1, 3, 3]
9/16 = 0.562500000 (+0.022462501), [0; 1, 1, 3, 2]
11/18 = 0.611111111 (-0.026148610), [0; 1, 1, 1, 1, 3]
5/9 = 0.555555556 (+0.029406945), [0; 1, 1, 4]
8/13 = 0.615384615 (-0.030422115), [0; 1, 1, 1, 1, 2]
6/11 = 0.545454545 (+0.039507955), [0; 1, 1, 5]
5/8 = 0.625000000 (-0.040037499), [0; 1, 1, 1, 2]